

Таблица `plant_images` связывает изображения с конкретными растениями по `plant_id`. Это позволяет хранить несколько изображений для одной карточки. Таблица `reviews` содержит `plant_id`, `admin_id`, `status` и `comment`, поэтому может фиксировать состояние проверки записи: `approved`, `pending` или другое значение, принятое в учебной модели.

В серверной части проекта также подготовлен файл `exoflowers-server/sql/schema.sql`, который описывает упрощенную таблицу `plants` для работы демонстрационного API. Backend реализован так, чтобы приложение могло запускаться как с PostgreSQL через `DATABASE_URL`, так и без базы данных через fallback-массив `data/plants.js`. Такой подход позволил сохранить работоспособность проекта во время разработки и постепенно переходить к полноценной базе данных.

3. Администрировал базу данных и описывал защиту информации в базе данных

Для локального варианта проекта база данных предполагается к развертыванию в PostgreSQL. Подключение к базе настраивается через переменную окружения `DATABASE_URL` в формате `postgres://user:password@localhost:5432/exoflowers`. Серверное приложение читает эту переменную в `db.js`, создает пул подключений `pg.Pool` и выполняет SQL-запросы через общий метод `query`.

Администрирование базы данных включает создание БД `exoflowers`, выполнение SQL-скрипта, проверку структуры таблиц, заполнение демонстрационными данными и проверку запросов `SELECT`, `INSERT`, `UPDATE`, `DELETE` и `SELECT` с `JOIN`. Эти операции были отражены в материалах проекта и позволяют проверить, что таблицы связаны корректно, а данные можно получать как отдельно, так и совместно через внешние ключи.

Для разграничения доступа в учебной модели предусмотрены роли пользователей. Обычный пользователь может работать с каталогом и просматривать сведения о растениях, а администратор связан с проверкой и управлением записями. На уровне базы это отражается полем `role` в таблице `users` и связью `reviews.admin_id` с `users.id`. В промышленной версии для безопасности необходимо хранить не открытые пароли, а хеши паролей, а также выдавать приложению отдельного пользователя БД с минимально необходимыми правами.

Защита целостности данных обеспечивается первичными и внешними ключами. Первичные ключи однозначно идентифицируют записи, а внешние ключи не позволяют связать растение с несуществующей категорией, изображение - с несуществующим растением, а отзыв - с несуществующим растением или администратором. Для предотвращения дублирования учетных записей используется уникальность `email`.

Для безопасной передачи данных между приложением и базой используется серверный backend как промежуточный слой: клиент не подключается к PostgreSQL напрямую, а отправляет HTTP-запросы к Express API. Это скрывает строку подключения от